[Euron] 머신러닝 스터디 9주차 복습과

조윤서 · 방금 전

0

캐글 노트북: Insurance prediction with five Regressor Models https://www.kaggle.com/code/touba7/insurance-prediction-with-five-regressor-models/input

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder

from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn import metrics
from sklearn.metrics import r2_score
from sklearn.model_selection import cross_val_score
%matplotlib inline

import warnings
warnings.filterwarnings("ignore")
```

Date overview

```
df = pd.read_csv('insurance.csv')
df.head(10)
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
5	31	female	25.740	0	no	southeast	3756.62160
6	46	female	33.440	1	no	southeast	8240.58960
7	37	female	27.740	3	no	northwest	7281.50560
8	37	male	29.830	2	no	northeast	6406.41070
9	60	female	25.840	0	no	northwest	28923.13692

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1338 non-null   int64  
 1   sex       1338 non-null   object 
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64  
 4   smoker    1338 non-null   object 
 5   region    1338 non-null   object 
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

```
df.describe()
```

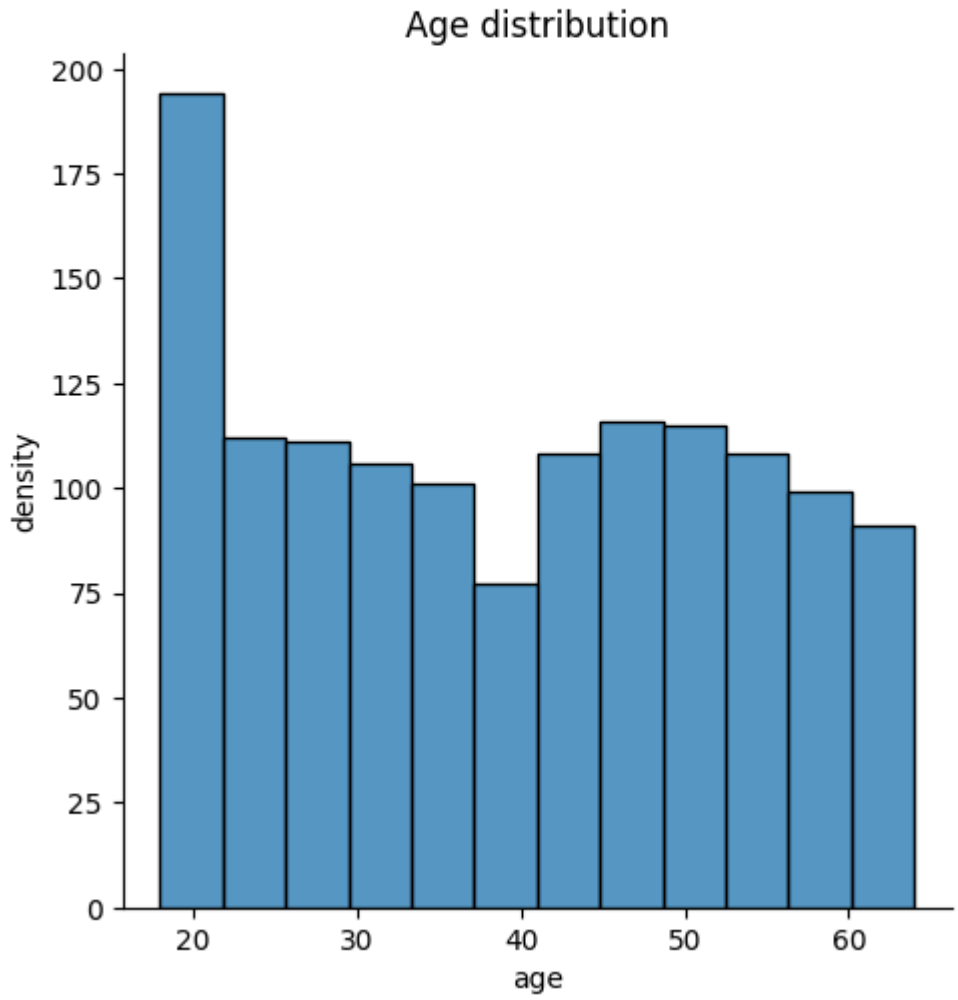
	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

- 1338개의 행과 7개의 변수로 구성
- 결측치 없음
- 숫자형 변수 4개와 범주형 변수 3개로 구성
- 평균 나이는 약 39.2세
- 평균 보험료는 약 13,279이고 최대 보험료는 약 63,770

Distribution Plot

나이 분포

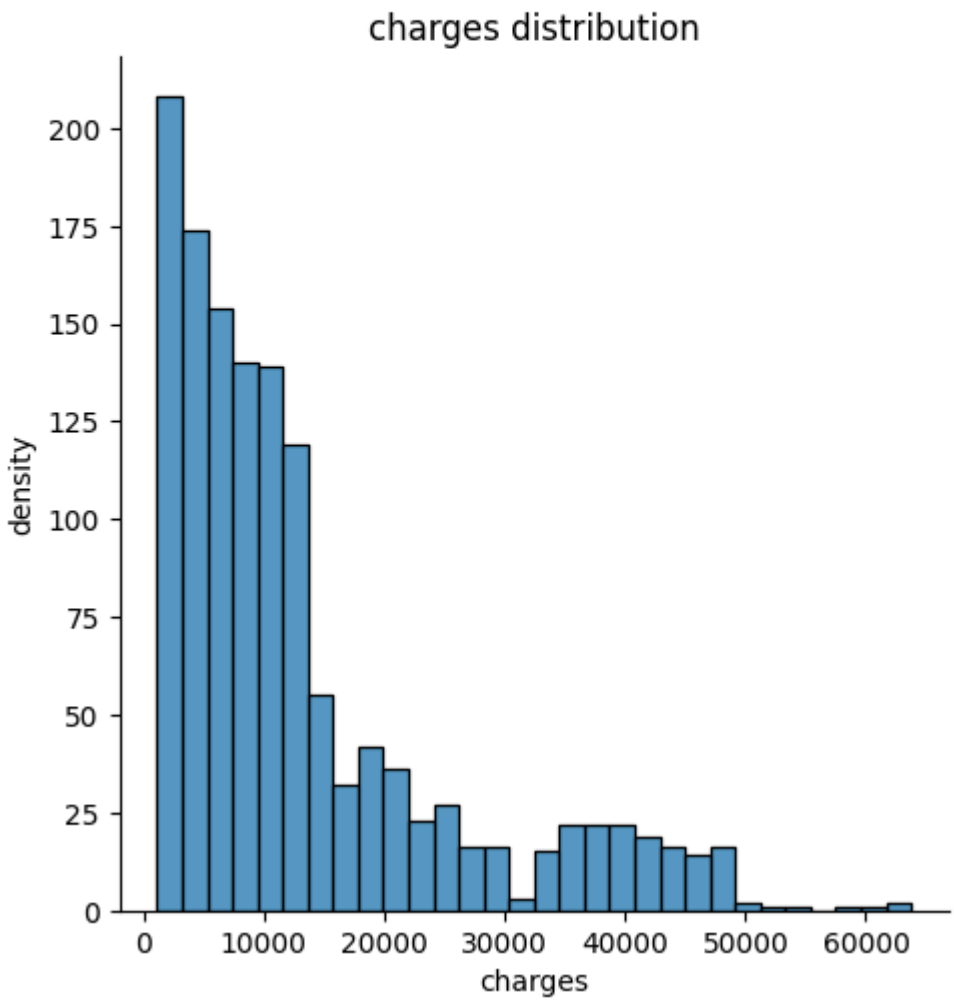
```
sns.displot(df['age'])
plt.title('Age distribution')
plt.ylabel('density')
plt.xlabel('age')
plt.show()
```



- 18세~21세 부근의 데이터가 가장 많음
- 나머지 연령대는 대체로 비슷한 빈도로 분포

보험료 분포

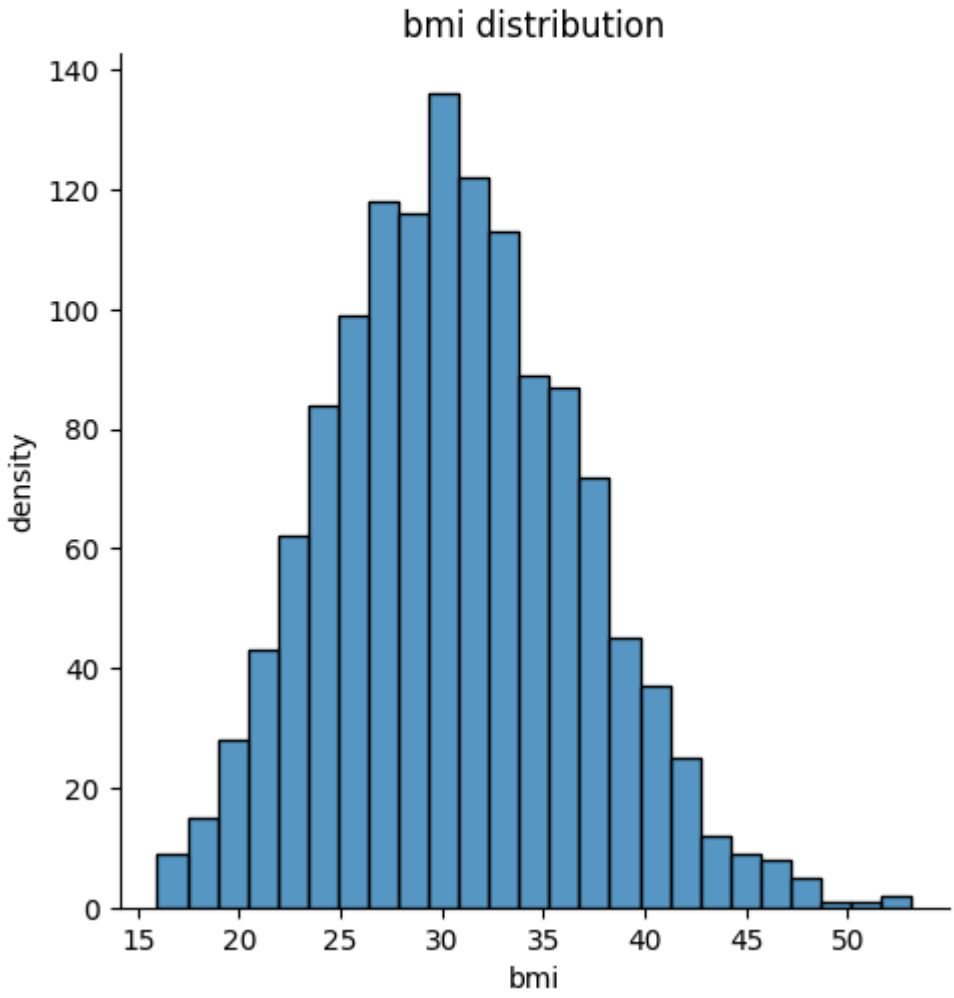
```
sns.displot(df['charges'])
plt.title('charges distribution')
plt.ylabel('density')
plt.xlabel('charges')
plt.show()
```



- 낮은 구간에 데이터가 집중되어있음
- 일부 고액 보험료 데이터가 존재

BMI 분포

```
sns.displot(df['bmi'])
plt.title('bmi distribution')
plt.ylabel('density')
plt.xlabel('bmi')
plt.show()
```



- 정규분포와 유사한 모양

Count Plot

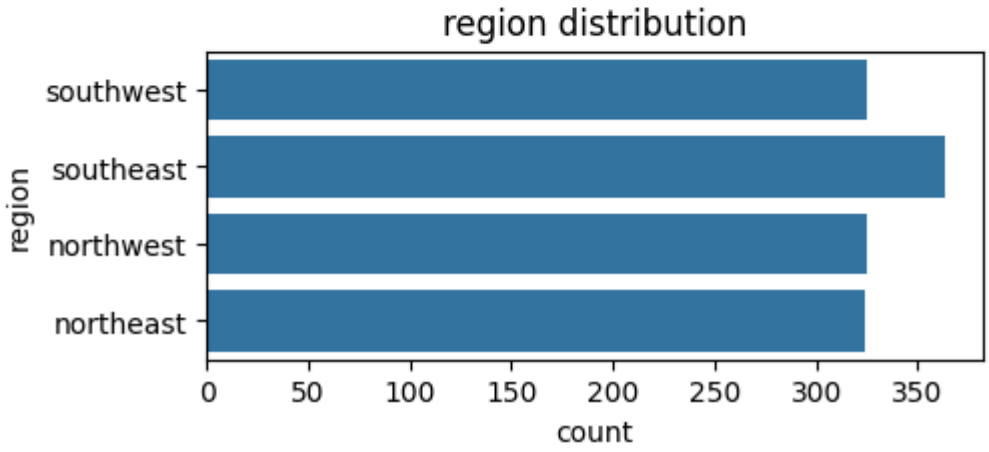
지역별 분포

```
df['region'].value_counts()
```

region	count
southeast	364
southwest	325
northwest	325
northeast	324

dtype: int64

```
fig=plt.figure(figsize=(5, 2))
sns.countplot(df['region'])
plt.title('region distribution')
plt.show()
```



- southeast 지역이 364명으로 가장 많음
- northeast 지역이 324명으로 가장 적음
- 비교적 균등하게 분포

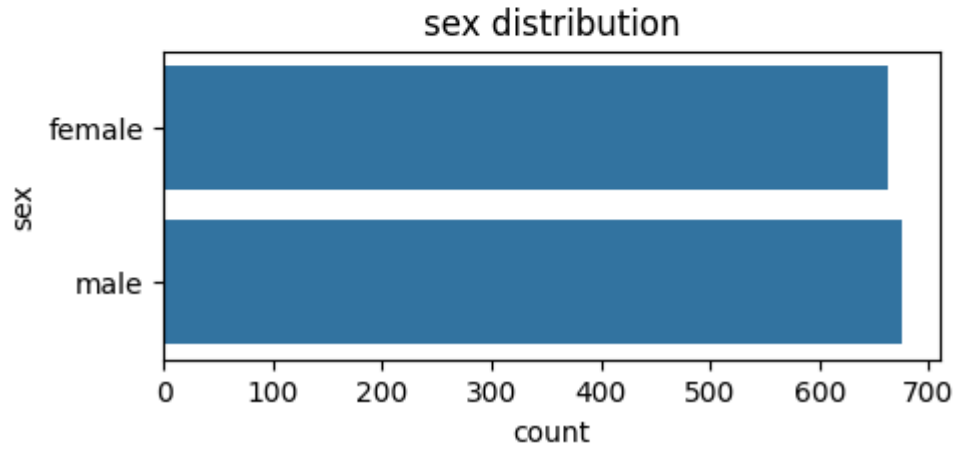
성별 분포

```
df['sex'].value_counts()
```

sex	count
male	676
female	662

dtype: int64

```
fig = plt.figure(figsize = (5,2))
sns.countplot(df['sex'])
plt.title('sex distribution')
plt.show()
```



- 남성이 여성보다 14명 더 많음
- 거의 균등하게 분포

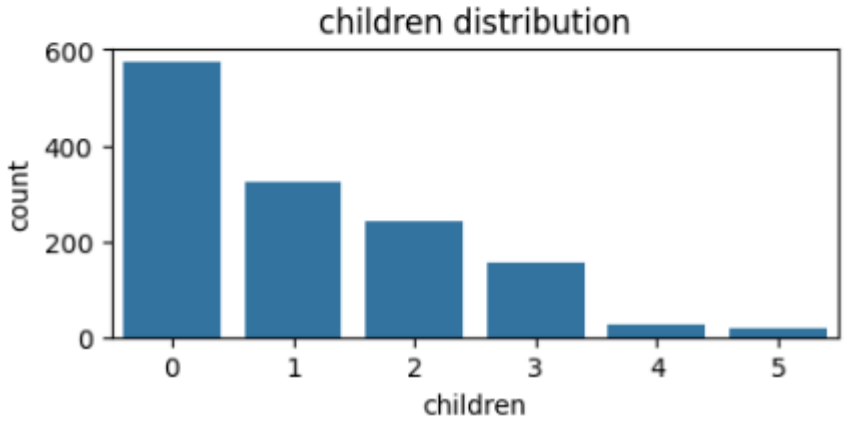
자녀 수 분포

```
df['children'].value_counts()
```

children	count
0	574
1	324
2	240
3	157
4	25
5	18

dtype: int64

```
plt.figure(figsize=(5, 2))
sns.countplot(data=df, x='children')
plt.title('children distribution')
plt.show()
```



- 자녀가 0명인 경우가 574명으로 가장 많음
- 전반적으로 자녀 수가 증가할 수록 데이터 수가 감소

Checking the correlation

보험료와 변수 간 상관관계

```
df['sex'] = pd.factorize(df['sex'])[0] + 1
df['region'] = pd.factorize(df['region'])[0] + 1
df['smoker'] = pd.factorize(df['smoker'])[0] + 1
corr = df.corr()
corr['charges'].sort_values(ascending=False)
```

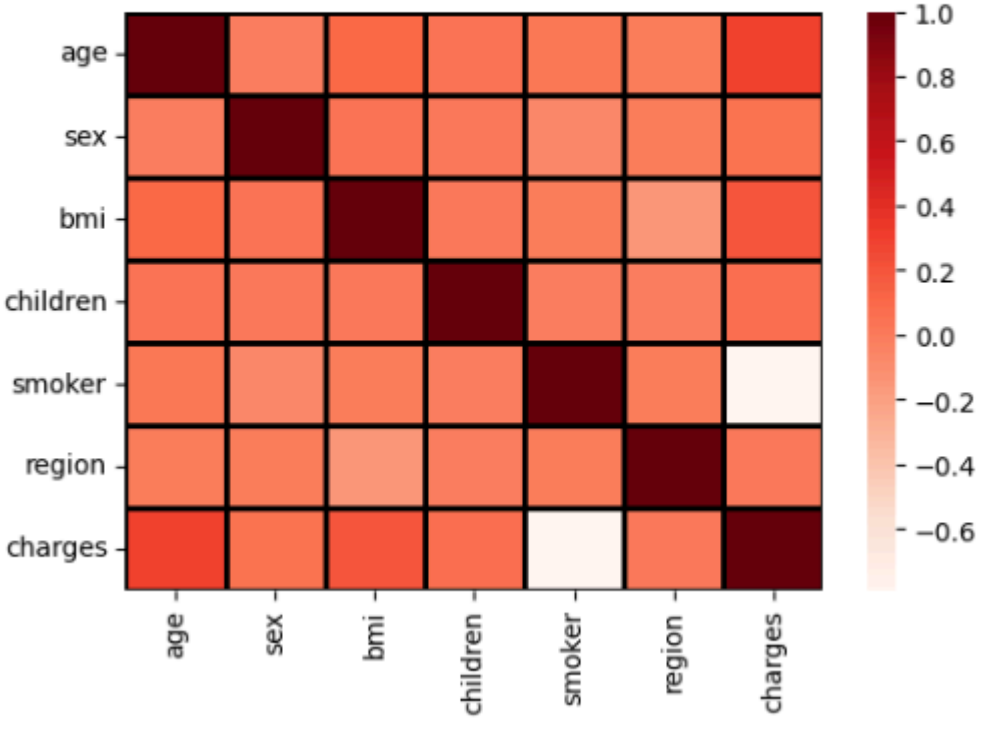
	charges
charges	1.000000
age	0.299008
bmi	0.198341
children	0.067998
sex	0.057292
region	0.006208
smoker	-0.787251

dtype: float64

- 보험료와 흡연 여부가 -0.787로 가장 강한 음의 상관관계를 보임
- 보험료와 BMI, age는 약한 양의 상관관계를 보임

시각화

```
fig = plt.figure(figsize = (6, 4))
data_ploting = df.corr(method= 'pearson')
sns.heatmap(data_ploting, cmap='Reds', linecolor='black', linewidths= 2 )
plt.show()
```



Modeling

Data Preprocession

```
X = df.drop('charges', axis = 1)
y = df['charges']
X_train, X_test, y_train, y_test= train_test_split(X, y, test_size=0.3, random_state=101)
```

```
scaler= StandardScaler()
scaler.fit(X_train)
X_train_scaled= scaler.transform(X_train)
X_test_scaled= scaler.transform(X_test)
```

- 학습용, 데이터용으로 분리한 뒤 데이터 표준화

Linear Regression Model

```
linear_reg_model= LinearRegression()
linear_reg_model.fit(X_train_scaled, y_train)

y_pred = linear_reg_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_li_reg= metrics.mean_absolute_error(y_test, y_pred)
MSE_li_reg = metrics.mean_squared_error(y_test, y_pred)
RMSE_li_reg =np.sqrt(MSE_li_reg)
pd.DataFrame([MAE_li_reg, MSE_li_reg, RMSE_li_reg],
             index=['MAE_li_reg', 'MSE_li_reg', 'RMSE_li_reg'],
             columns=['Metrics'])
```

	Metrics
MAE_li_reg	3.990250e+03
MSE_li_reg	3.353013e+07
RMSE_li_reg	5.790521e+03

- MAE는 3990, RMSE는 5791

```
scores = cross_val_score(linear_reg_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.88791389 0.85653048 0.84404195 0.87198372 0.84417492]
```

```
r2_score(y_test, linear_reg_model.predict(X_test_scaled))
```

```
0.7613126015198817
```

- 5-Fold 교차검증
- R^2는 약 0.761

Gradient Boosting Regressor Model

```
Gradient_model = GradientBoostingRegressor()
Gradient_model.fit(X_train_scaled, y_train)

y_pred = Gradient_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_gradient= metrics.mean_absolute_error(y_test, y_pred)
MSE_gradient = metrics.mean_squared_error(y_test, y_pred)
RMSE_gradient =np.sqrt(MSE_gradient)
pd.DataFrame([MAE_gradient, MSE_gradient, RMSE_gradient],
             index=['MAE_gradient', 'MSE_gradient', 'RMSE_gradient'],
             columns=['Metrics'])
```

	Metrics
MAE_gradient	2.528851e+03
MSE_gradient	2.110725e+07
RMSE_gradient	4.594262e+03

- MAE는 약 2529, RMSE는 약 4.594
- 선형회귀보다 더 나은 예측 성능을 보임

```
scores = cross_val_score(Gradient_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.94584072 0.91427606 0.91679265 0.92440285 0.91717029]
```

```
r2_score(y_test, Gradient_model.predict(X_test_scaled))
```

```
0.849746079970233
```

- R^2은 약 0.8497로 설명력이 선형회귀보다 좋음

XGB Regressor Model

```
XGB_model =XGBRegressor()
XGB_model.fit(X_train_scaled, y_train);

y_pred = XGB_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_XGB= metrics.mean_absolute_error(y_test, y_pred)
MSE_XGB = metrics.mean_squared_error(y_test, y_pred)
RMSE_XGB =np.sqrt(MSE_XGB)
pd.DataFrame([MAE_XGB, MSE_XGB, RMSE_XGB],
             index=['MAE_XGB', 'MSE_XGB', 'RMSE_XGB'],
             columns=['Metrics'])
```

	Metrics
MAE_XGB	3.198314e+03
MSE_XGB	3.061846e+07
RMSE_XGB	5.533395e+03

- MAE는 약 3198, RMSE는 약 5533
- 선형 회귀보다는 성능이 좋지만 Gradient Boosting보다는 오차가 큼

```
scores = cross_val_score(XGB_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.90787759 0.88795747 0.91289956 0.89459789 0.88672783]
```

```
r2_score(y_test, XGB_model.predict(X_test_scaled))
```

```
0.7820396260594861
```

- R^2는 약 0.782로 선형회귀보다 우수하지만 Gradient Boosting보다는 낮음

Decision Tree Regressor Model

```
tree_reg_model =DecisionTreeRegressor()
tree_reg_model.fit(X_train_scaled, y_train);

y_pred = tree_reg_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_tree_reg= metrics.mean_absolute_error(y_test, y_pred)
MSE_tree_reg = metrics.mean_squared_error(y_test, y_pred)
RMSE_tree_reg =np.sqrt(MSE_tree_reg)
pd.DataFrame([MAE_tree_reg, MSE_tree_reg, RMSE_tree_reg],
             index=['MAE_tree_reg', 'MSE_tree_reg', 'RMSE_tree_reg'],
             columns=['Metrics'])
```

	Metrics
MAE_tree_reg	3.462211e+03
MSE_tree_reg	4.963572e+07
RMSE_tree_reg	7.045262e+03

- MAE는 약 3462, RMSE는 약 7045
- 선형 회귀보다 MAE는 낮지만 RMSE는 높음 -> 큰 오차에 취약한 모델

```
scores = cross_val_score(tree_reg_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.83953516 0.8102413  0.8665669  0.86761853 0.84565665]
```

```
r2_score(y_test, tree_reg_model.predict(X_test_scaled))
```

```
0.6466634327315981
```

- R^2은 약 0.647로 가장 낮은 설명력을 가지고 있음
- 과적합 가능성 존재

Random Forest Regressor Model

```
forest_reg_model =RandomForestRegressor()
forest_reg_model.fit(X_train_scaled, y_train);

y_pred = forest_reg_model.predict(X_test_scaled)
y_pred = pd.DataFrame(y_pred)
MAE_forest_reg= metrics.mean_absolute_error(y_test, y_pred)
MSE_forest_reg = metrics.mean_squared_error(y_test, y_pred)
RMSE_forest_reg =np.sqrt(MSE_forest_reg)
pd.DataFrame([MAE_forest_reg, MSE_forest_reg, RMSE_forest_reg],
             index=['MAE_forest_reg', 'MSE_forest_reg', 'RMSE_forest_reg'],
             columns=['Metrics'])
```

	Metrics
MAE_forest_reg	2.839379e+03
MSE_forest_reg	2.464995e+07
RMSE_forest_reg	4.964871e+03

- MAE는 약 2839, RMSE는 약 4965

```
scores = cross_val_score(forest_reg_model, X_train_scaled, y_train, cv=5)
print(np.sqrt(scores))
```

```
[0.93834357 0.90034112 0.91987162 0.91465326 0.90624011]
```

```
r2_score(y_test, forest_reg_model.predict(X_test_scaled))
```

```
0.8245270225560383
```

- R^2은 약 0.825
- 비교적 우수한 성능을 보이지만 Gradient Boosting이 여전히 가장 좋은 성능을 보임

Gradient Boosting > Random Forest > XGBoost > Linear Regression > Decision Tree 순으로 높은 성능을 가짐



조윤서

이전 포스트

0개의 댓글

댓글을 작성하세요

댓글 작성

Powered by Stellate